

Guia Prático de

# LogQL

Loki Query Language

Este guia foi criado para ajudar qualquer usuário — mesmo sem experiência técnica — a entender e utilizar o LogQL para consultar logs centralizados no Grafana Loki. Você aprenderá a filtrar, analisar e extrair métricas de logs com exemplos práticos e diretos.

Conteúdo deste Guia

|    |                                             |
|----|---------------------------------------------|
| 01 | O que é LogQL e como funciona no Loki       |
| 02 | Log Stream Selector — encontrando seus logs |
| 03 | Pipeline de Filtros — refinando resultados  |
| 04 | Metric Queries — extraindo métricas de logs |

## 01 — O que é LogQL?

LogQL é a linguagem de consulta do **Grafana Loki**, uma solução de agregação de logs altamente escalável. Diferente do Elasticsearch, o Loki **não indexa o conteúdo** dos logs — apenas os rótulos (labels). Isso torna o armazenamento muito mais barato e eficiente.

Uma query LogQL é sempre composta por duas partes: o **Stream Selector** (obrigatório) que seleciona de qual fonte vêm os logs, e o **Pipeline** (opcional) que filtra e transforma as linhas.

### ■ Anatomia de uma query LogQL

```
{app="pagamentos"} |= "ERROR"
```

```
# ■■■ Stream Selector ■■■ ■■ Pipeline ■■
```

```
# Busca logs do app 'pagamentos' que contenham a palavra ERROR
```

## 02 — Log Stream Selector

O Stream Selector usa `{ }` para identificar qual conjunto de logs você quer consultar, baseado nos labels configurados no Loki. É a parte mais importante da query — sem ele, nada funciona.

### Operadores de seleção:

| Operador        | Significado          | Exemplo                          |
|-----------------|----------------------|----------------------------------|
| <code>=</code>  | Igual exato          | <code>{app="api-gateway"}</code> |
| <code>!=</code> | Diferente de         | <code>{env!="prod"}</code>       |
| <code>=~</code> | Regex: contém padrão | <code>{job=~"worker-.*"}</code>  |
| <code>!~</code> | Regex: não contém    | <code>{job!~"test-.*"}</code>    |

### ■ Combinando múltiplos labels (AND implícito)

```
{app="checkout", env="prod", region="br-east"}
```

```
# Retorna logs do checkout em produção na região br-east
```

■ **Dica:** Use sempre o menor número de labels que identifique unicamente seu serviço. Muitos labels combinados podem retornar zero resultados se um deles não existir.

## 03 — Pipeline de Filtros

O pipeline é uma cadeia de operações aplicadas linha a linha nos logs selecionados. Cada etapa começa com `|` (pipe) e processa a saída da etapa anterior.

### Filtros de texto — `|=` `!=` `|~` `!~`

| Operador        | O que faz                  | Exemplo                       |
|-----------------|----------------------------|-------------------------------|
| <code> =</code> | Linha contém texto (exato) | <code> = "timeout"</code>     |
| <code>!=</code> | Linha NÃO contém texto     | <code>!= "healthcheck"</code> |
| <code> ~</code> | Linha bate com regex       | <code> ~ "ERR(OR)?"</code>    |
| <code>!~</code> | Linha NÃO bate com regex   | <code>!~ "DEBUG TRACE"</code> |

#### ■ Encadeando filtros — logs de erro excluindo health checks

```
{app="api"} |= "ERROR" != "healthcheck" !~ "DEBUG|TRACE"
```

```
# Passo 1: seleciona logs do app 'api'
# Passo 2: mantém só linhas com 'ERROR'
# Passo 3: remove linhas com 'healthcheck'
# Passo 4: remove linhas de DEBUG ou TRACE
```

### Parsers — extraindo campos dos logs

Parsers transformam o texto bruto do log em campos estruturados que podem ser filtrados e usados em métricas:

| Parser                 | Uso ideal                      | Exemplo de uso                                               |
|------------------------|--------------------------------|--------------------------------------------------------------|
| <code>  json</code>    | Logs em formato JSON           | <code>  json   level="error"</code>                          |
| <code>  logfmt</code>  | Logs key=value (estilo Go)     | <code>  logfmt   duration &gt; 1s</code>                     |
| <code>  pattern</code> | Logs com formato fixo          | <code>  pattern "&lt;ip&gt; - &lt;user&gt; &lt;_&gt;"</code> |
| <code>  regexp</code>  | Extração via regex nomeada     | <code>  regexp "(?P&lt;code&gt;[0-9]{3})"</code>             |
| <code>  unpack</code>  | Logs empacotados pelo Promtail | <code>  unpack</code>                                        |

#### ■ Usando `| json` para filtrar campos específicos

```
{app="pagamentos"} | json | status_code >= 500
```

```
# Log original: {"status_code":503,"msg":"upstream timeout","trace":"abc123"}
# | json → extrai campos automaticamente
# status_code >= 500 → filtra só os erros 5xx
```

■ **Dica:** Use `| json` sempre que seus logs forem estruturados em JSON — é o parser mais poderoso e permite filtrar qualquer campo sem regex.

## 04 — Metric Queries

Além de buscar linhas de log, o LogQL pode **gerar métricas a partir dos logs**. Isso é muito útil para dashboards no Grafana — por exemplo, contar erros por segundo ou calcular latência média a partir de logs de acesso.

Funções de métrica essenciais:

| Função          | O que faz                    | Exemplo                                                  |
|-----------------|------------------------------|----------------------------------------------------------|
| count_over_time | Conta linhas no período      | count_over_time({app='api'}[5m])                         |
| rate            | Linhas por segundo           | rate({app='api'}  = 'ERROR'[5m])                         |
| bytes_over_time | Volume de dados no período   | bytes_over_time({job='nginx'}[1h])                       |
| bytes_rate      | Bytes por segundo            | bytes_rate({job='nginx'}[5m])                            |
| sum_over_time   | Soma campo numérico extraído | sum_over_time({app='api'}   json   unwrap duration [5m]) |

## Exemplos Completos e Práticos

Taxa de erros por segundo nos últimos 5 minutos

```
rate({app="checkout"} |= "ERROR" [5m])
```

# Resultado: 3.2 erros/s → serviço com alto volume de erros

Contar erros HTTP 5xx agrupados por serviço

```
sum by (app) (
count_over_time(
{namespace="producao"} | json | status_code >= 500 [10m]
)
)
```

# Resultado: app=pagamentos → 142, app=auth → 5

Top 5 serviços com mais linhas de log (volume)

```
topk(5,
sum by (app) (rate({namespace="producao"}[5m]))
)
```

# Identifica quais apps estão gerando mais logs

Latência média extraída de logs JSON com | unwrap

```
avg_over_time(
{app="api-gateway"} | json | unwrap response_time_ms [5m]
) by (app)
```

# Log: {"app":"api-gateway","response\_time\_ms":142,"status":200}  
# Resultado: média de 142ms de latência nos últimos 5 min

### Referência Rápida — Intervalos de Tempo

| s     | Segundos | m    | Minutos | h    | Horas  | d    | Dias   | w    | Semanas |
|-------|----------|------|---------|------|--------|------|--------|------|---------|
| [30s] | 30 seg   | [5m] | 5 min   | [1h] | 1 hora | [7d] | 7 dias | [4w] | 4 sem   |

■ **Dica:** Para praticar LogQL, acesse o **Grafana Play** em [play.grafana.org](https://play.grafana.org) — ele já tem um Loki com dados reais para você explorar sem instalar nada.